

Enterprise ETL Pipeline & Data Warehouse Synchronizer

Technical Documentation and Operations Guide

Version 1.0 | 8 September 2026

Purpose

This document explains how to install, configure, run, extend, test, and operate the Enterprise ETL Pipeline & Data Warehouse Synchronizer. The application collects operational data from Stripe and Salesforce, retains raw source payloads, validates and transforms records, and is designed to load the results incrementally into PostgreSQL or Snowflake through Apache Airflow.

Intended audience

Python developers, data engineers, project evaluators, and operations users responsible for API credentials, Airflow, or warehouse access.

System Overview

Pipeline flow: Stripe / Salesforce APIs → raw JSON landing area → validation → Pandas transformation → staging layer → warehouse upsert → analytics tables.

The supplied starter implementation completes extraction, local raw-data persistence, validation, transformation, Airflow scheduling, Docker-based development setup, and a basic automated test. The final physical warehouse load is intentionally represented as an extension point so it can be adapted to PostgreSQL or Snowflake schemas.

Repository Layout

```
enterprise_etl_project/
├── .env.example           # Credential and environment template
├── docker-compose.yml     # PostgreSQL and Airflow local services
├── requirements.txt       # Python dependencies
├── README.md             # Quick-start project overview
├── dags/
│   └── enterprise_etl.py  # Daily Airflow DAG
├── etl/
│   └── pipeline.py        # Extraction and transformation functions
├── tests/
│   └── test_pipeline.py   # Pytest transformation test
```

Prerequisites

- Python 3.10 or later
- Docker Desktop or Docker Engine with Docker Compose
- A Stripe account and a test-mode Stripe secret key
- A Salesforce organization with API access and OAuth configured
- Optional: AWS account and S3 bucket for durable raw-data storage
- Optional: PostgreSQL or Snowflake warehouse credentials for the load layer

Installation

1. Create the working copy and enter the project directory.

```
cd enterprise_etl_project
```

2. Create a private environment file from the template.

```
cp .env.example .env
```

3. Install Python dependencies in a virtual environment if you are running modules outside Docker.

```
python -m venv .venv
# macOS/Linux
source .venv/bin/activate
# Windows PowerShell
# .venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

4. Start the local PostgreSQL and Airflow services.

```
docker compose up -d
```

5. Run unit tests.

```
pytest
```

6. Open Airflow in a browser at <http://localhost:8080>, then enable the DAG named `enterprise_etl_daily`.

Configuration

Configuration is supplied through environment variables. Never commit the .env file to a repository. Keep .env.example safe to commit because it contains placeholders only.

Environment variables

Variable	Required	Description
STRIPE_API_KEY	Yes	Stripe secret key. Begin with a sk_test_ key during development.
SALESFORCE_INSTANCE_URL	Yes	Salesforce org base URL, for example https://your-org.my.salesforce.com.
SALESFORCE_ACCESS_TOKEN	Temporary	A short-lived token for initial testing only.
SALESFORCE_CLIENT_ID	Recommended	OAuth client/consumer key for a production client-credentials implementation.
SALESFORCE_CLIENT_SECRET	Recommended	OAuth client/consumer secret. Store only in a secret manager or private environment.
DATABASE_URL	For PostgreSQL load	SQLAlchemy database connection string.
S3_BUCKET	Optional	Destination bucket for raw JSON archival.
AWS_REGION	Optional	Region hosting the S3 bucket.

Example .env

```
STRIPE_API_KEY=sk_test_replace_with_your_key
SALESFORCE_INSTANCE_URL=https://your-org.my.salesforce.com
SALESFORCE_CLIENT_ID=your_consumer_key
SALESFORCE_CLIENT_SECRET=your_consumer_secret
DATABASE_URL=postgresql+psycopg2://etl:etl@postgres:5432/warehouse
S3_BUCKET=etl-raw
AWS_REGION=ap-south-1
```

For scheduled production runs, replace the static SALESFORCE_ACCESS_TOKEN approach with OAuth client credentials. The job should obtain a fresh access token at runtime and avoid persisting it in source code, DAG files, or version control.

Extraction Modules

Stripe extraction

The function `fetch_stripe_payments(api_key, since)` sends authenticated requests to Stripe payment-intent endpoints. It requests up to 100 records per page and continues when the API indicates that additional pages are available. The optional `since` value is intended for incremental extraction using a source timestamp.

Salesforce extraction

The function `fetch_salesforce_accounts(instance_url, token, soql)` runs a SOQL query against the Salesforce REST API. When Salesforce returns `nextRecordsUrl`, the function calls that relative URL until all pages are collected. Configure the SOQL selection to retrieve only fields required downstream and to use an updated timestamp filter for incremental synchronization.

Raw landing data

`write_raw(records, path)` serializes the unmodified extracted records as JSON. In local development it writes to a file path. For production, replace or extend this method to write a date-partitioned object such as `s3://bucket/source=stripe/date=YYYY-MM-DD/run_id=/payload.json`.

Transformation and Validation

The Payment Pydantic model validates id, non-negative amount, currency, created timestamp, and status. Invalid source records are logged and excluded. The transform_payments function then creates a Pandas DataFrame, computes amount_major from the minor-unit amount, converts created UNIX timestamps into UTC timestamps, and selects the normalized output columns.

```
Input record
{"id":"pi_1", "amount":1250, "currency":"usd", "created":1700000000, "status":"succeeded"}

Normalized fields
id | amount | amount_major | currency | status | created_at
pi_1 | 1250 | 12.50 | usd | succeeded | UTC timestamp
```

Airflow Operations

DAG: enterprise_etl_daily

The included DAG uses a daily schedule, a fixed start date, and catchup=False. Its task sequence is extract → transform → load. Airflow creates task logs, records task state, and provides the operational control plane for retries and manual reruns.

Manual run

```
# From the Airflow UI: enable the DAG, then select Trigger DAG.
# Or, inside the Airflow container:
docker compose exec airflow airflow dags trigger enterprise_etl_daily
```

Recommended task configuration

Add retries, retry_delay, execution timeout, failure notifications, and a logical run identifier. For example, give API calls bounded timeouts, use exponential backoff on transient 429/5xx errors, and configure alerting when the final load task fails.

Incremental synchronization

Create an etl_state table that stores a high-water mark per source and entity. After a successful load, write the greatest processed created_at or updated_at value. The next run should query only data after the saved watermark, with an overlap window to protect against delayed source updates. The final upsert must be idempotent so overlap records are updated instead of duplicated.

Warehouse Load Design

The current load task is a placeholder. Implement the production load as a two-stage operation: load the transformed DataFrame into a staging table, then merge it into a target table using the source system identifier as the natural key.

PostgreSQL example

```
CREATE TABLE IF NOT EXISTS fact_stripe_payment (
  id TEXT PRIMARY KEY,
  amount INTEGER NOT NULL,
  amount_major NUMERIC(18,2) NOT NULL,
  currency TEXT NOT NULL,
  status TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL,
  loaded_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO fact_stripe_payment (id, amount, amount_major, currency, status, created_at)
VALUES (:id, :amount, :amount_major, :currency, :status, :created_at)
ON CONFLICT (id) DO UPDATE SET
  amount = EXCLUDED.amount,
  amount_major = EXCLUDED.amount_major,
  currency = EXCLUDED.currency,
  status = EXCLUDED.status,
  created_at = EXCLUDED.created_at;
```

For Snowflake, bulk-load a staging table and use MERGE INTO target USING staging ON target.id = staging.id. Record pipeline run metadata, source watermark, input count, rejected count, and loaded count for auditability.

Testing

Run the test suite with pytest. The included test checks that a payment amount of 1250 is transformed to an amount_major of 12.5 and verifies the currency output. Add tests for empty inputs, malformed records, API pagination, HTTP failures, duplicate IDs, watermark boundaries, and idempotent database loads.

```
pytest -q
```

Security Checklist

- Keep `.env` out of Git by adding it to `.gitignore`.
- Use Stripe test keys for development and restricted keys in deployed services.
- Use a dedicated Salesforce integration user with least-privilege object and field permissions.
- Use Salesforce OAuth client credentials and obtain short-lived access tokens at runtime.
- Store production secrets in Airflow Connections, AWS Secrets Manager, or a comparable service.
- Rotate secrets immediately if they appear in logs, screenshots, source code, or commits.
- Avoid logging Authorization headers, access tokens, raw personally identifiable information, or full payment details.
- Encrypt raw data at rest and restrict S3 bucket and database access using IAM roles.

Troubleshooting

Issue	Likely cause	Resolution
401/403 from Stripe	Invalid, expired, or restricted key	Confirm test/live mode, regenerate the key, and verify permissions.
401 from Salesforce	Expired token or OAuth configuration issue	Obtain a new token; verify client credentials, assigned user, and OAuth scopes.
Salesforce query fails	Object/field permission or SOQL problem	Test the SOQL in Salesforce tools and grant the integration user access.
Duplicate warehouse rows	Non-idempotent load or missing natural key	Use id as a primary key and ON CONFLICT/MERGE logic.
DAG not visible	DAG folder mount or Python import failure	Check Docker volume mapping and inspect Airflow scheduler logs.
Pipeline is slow	Large full refresh or API rate limit	Use watermark-based incremental extraction and batch warehouse writes.

Production Readiness

Before production deployment, complete the physical warehouse-load implementation, establish a raw-data retention policy, create dashboards for task success/failure and record counts, add alerting, establish backup/recovery procedures, validate API rate limits, use managed secret storage, and run an initial backfill under controlled monitoring. Document ownership of each data source and define service-level expectations for freshness and recovery time.